GANNAN NORMAL UNIVERSITY

STUDENT MANAGEMENT SYSTEM

By

Arman Rana Muhammad

中国 名字：杨星

Student ID: 202337022

Under the guidance of 彭建荣

Submitted to Computer Science and Technology
Final Project Documentation

贛南师范大学，赣州，江西

TITLE OF PROJECT REPORT

SOFTWARE TESTING FINAL PROJECT REPORT

Submitted by

ARMAN RANA MUHAMMAD

CHINESE NAME : 杨星

STUDENT ID: 202337022

SUPERVISOR : 彭建荣

赣南师范大学

Date ： 27/12/2025

Declaration

I, Arman Rana Muhammad, Student ID: 202337022, I hereby declare that this project entitled **Software Testing Final Project Report on Student Management System (SMS)** is my original work carried out under the guidance of my supervisor.

This report has not been submitted, either in whole or in part, to any other university or institution for the award of any degree or diploma.All sources of information used in this report have been properly acknowledged.

Name: Arman Rana Muhammad

Student ID: 202337022

Date: 27/12/2025

Abstract

This report presents a comprehensive software testing study conducted on the Student Management System (SMS), a web-based application developed to support academic and administrative activities in educational institutions.The primary objective of this project is to ensure that the system functions correctly, securely, and reliably according to specified requirements.

Various testing techniques were applied, including functional testing, black box testing, boundary value analysis, integration testing, system testing, regression testing, user acceptance testing, white box testing, and basic performance testing. Testing was performed across multiple user roles, including Admin, Teacher, and Student, to verify role-based access control and data integrity.

The testing environment consisted of Django as the backend framework, PostgreSQL as the database, and Google Chrome as the testing browser, with deployment managed through Docker. A wide range of test cases were designed and executed to validate all major system modules such as user authentication, course management, enrollment, assignments, attendance, examinations, payments, and contact handling.

The results of the testing process indicate that the Student Management System meets its functional requirements and performs reliably under normal usage conditions. All critical defects were identified and resolved, and the system was found to be stable and suitable for academic deployment.

List of Tables

Table 1: User Authentication and Authorization Test Cases

Table 2: Department Management Test Cases

Table 3: Course Management Test Cases

Table 4: Student Enrollment Test Cases

Table 5: Assignment and Coursework Management – Teacher Creates Assignment Test Cases

Table 6: Assignment and Coursework Management – Student Submits Assignment Test Cases

Table 7: Assignment and Coursework Management – Teacher Grades Assignment Test Cases

Table 8: Attendance Management Test Cases

Table 9: Exam Creation Test Cases

Table 10: Exam Scheduling Test Cases

Table 11: Exam Result Entry and Viewing Test Cases

Table 12: Payment Management Test Cases

Table 13: Contact Message Handling Test Cases

Functional Testing Tables

Table 14: Functional Testing – User Authentication and Authorization Results

Table 15: Functional Testing – Department Management Results

Table 16: Functional Testing – Course Management Results

Table 17: Functional Testing – Student Enrollment Results

Table 18: Functional Testing – Assignment and Coursework Management Results

Table 19: Functional Testing – Attendance Management Results

Table 20: Functional Testing – Exam and Result Management Results

Table 21: Functional Testing – Payment Management Results

Table 22: Functional Testing – Contact Message Handling Results

Integration Testing Tables

Table 23: Student–Course Integration Test Cases

Table 24: Course–Teacher Integration Test Cases

Table 25: Assignment–Submission Integration Test Cases

Table 26: Exam–Result Integration Test Cases

Table 27: Payment–Student Integration Test Cases

System Testing Tables

Table 28: System Test Case Table

White Box Testing Tables

Table 29: White Box Test Cases – Admin Access Control

Table 30: White Box Test Cases – Enrollment Duplicate Prevention

Table 31: White Box Test Cases – Assignment Due Date Validation

Table 32: White Box Test Cases – Marks Boundary Validation

Table 33: White Box Test Cases – Course Visibility Restriction

Table 34: Jazzmin Admin UI Test Checks

Performance Testing Tables

Table 35: Authentication and Authorization Performance Test Results

Table 36: Django Admin Core Performance Test Results

Table 37: Enrollment Module Performance Test Results

Table 38: Assignment and Submission Performance Test Results

Table 39: Attendance Performance Test Results

Table 40: Exam and Result Performance Test Results

Table 41: Payment Performance Test Results

Table 42: Contact Message Handling Performance Test Results

Table 43: Stability and Concurrency Test Results

Table 44: Defect Summary

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION AND SYSTEM OVERVIEW

1.1 Introduction to Software Testing

1.2 System Overview

CHAPTER 2: TESTING OBJECTIVES, SCOPE AND METHODOLOGY

2.1 Testing Objectives

2.2 Testing Scope

2.3 Testing Methodology

CHAPTER 3: TESTING ENVIRONMENT

3.1 Hardware Environment

3.2 Software Environment

CHAPTER 4: FUNCTIONAL TESTING

4.1 User Authentication and Authorization

4.2 Department Management

4.3 Course Management

4.4 Student Enrollment

4.5 Assignment and Coursework Management

4.6 Attendance Management

4.7 Exam and Result Management

4.8 Payment Management

4.9 Contact Message Handling

CHAPTER 5: INTEGRATION TESTING

5.1 Introduction to Integration Testing

5.2 Integration Testing Approach

5.3 Student–Course Integration

5.4 Course–Teacher Integration

5.5 Assignment–Submission Integration

5.6 Exam–Result Integration

5.7 Payment–Student Integration

5.8 Integration Testing Summary

CHAPTER 6: SYSTEM TESTING

6.1 Introduction to System Testing
6.2 Objectives of System Testing
6.3 System Testing Environment
6.4 System Testing Strategy
6.5 End-to-End Workflow Description
6.6 System Test Case Table
6.7 System Testing Results
6.8 Issues Identified During System Testing
6.9 System Testing Summary

CHAPTER 7: REGRESSION AND USER ACCEPTANCE TESTING

7.1 Regression Testing
7.2 User Acceptance Testing (UAT)

CHAPTER 8: WHITE BOX TESTING

8.1 Introduction
8.2 White Box Testing Objectives
8.3 White Box Testing Techniques Used
8.4 Django Admin Logic and White Box Test Cases
8.5 Jazzmin (Admin UI Theme) Testing Notes
8.6 White Box Testing Summary

CHAPTER 9: PERFORMANCE TESTING

9.1 Introduction
9.2 Objectives
9.3 Performance Testing Environment
9.4 Tools and Measurement Method
9.5 Performance Test Cases
9.6 Performance Testing Summary

CHAPTER 10: TEST RESULTS AND DEFECT SUMMARY

10.1 Test Execution Summary
10.2 Defect Summary
10.3 Overall Test Result

CHAPTER 11: LIMITATIONS, FUTURE RECOMMENDATIONS AND CONCLUSION

11.1 Limitations of Testing
11.2 Future Testing Recommendations
11.3 Conclusion

CHAPTER 1: INTRODUCTION AND SYSTEM OVERVIEW

1.1 Introduction to Software Testing

Software testing is an essential phase of the Software Development Life Cycle (SDLC). It ensures that a software system functions according to requirements, is reliable, and meets user expectations. Testing helps identify defects early, reduce system failures, and improve overall software quality.

This report presents a detailed software testing study conducted on the **Student Management System (SMS)**. The system is a web-based application developed using the Django framework and PostgreSQL database. The purpose of this report is to document the testing objectives, scope, methodologies, test cases, defects, and conclusions derived from the testing process.

1.2 System Overview

The Student Management System (SMS) is designed to support academic and administrative operations in colleges and universities. The system is role-based and supports three main users:

1.Admin

2.Teacher

3.Student

Major Functional Modules:

1. User Authentication & Authorization
2. Department Management
3. Course Management
4. Student Enrollment
5. Assignment & Coursework Management
6 . Attendance Management
7. Exam & Result Management
8. Payment Management
9. Contact Message Handling

The system ensures secure access, data integrity, and controlled workflows.

Testing of Major Functional Modules:

1) User Authentication & Authorization — Test Cases

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
AUTH-01	All	Valid login	User exists	Open login → enter creds → Login	user=student1 pass=Correct123	Redirect to correct dashboard	Black box
AUTH-02	All	Invalid password	User exists	Enter wrong password	pass=Wrong123	"Invalid credentials" shown, stay on login	Black box
AUTH-03	All	Invalid username	None	Enter unknown username	user=unknown	Error message shown	Black box
AUTH-04	All	Empty fields	None	Click Login without input	blank	Validation shown	Black box
AUTH-05	All	Logout works	Logged in	Click Logout	—	Session ends, redirected to login/landing	Functional
AUTH-06	Student	Access admin URL blocked	Logged in as student	Open /admin	—	Access denied / redirected	Authorization
AUTH-07	Teacher	Access admin URL blocked	Logged in as teacher	Open /admin	—	Access denied / redirected	Authorization
AUTH-08	Admin	Admin can access admin panel	Admin exists	Login → open /admin	admin creds	Admin panel loads	Functional
AUTH-09	All	Session timeout	Logged in	Idle / expire session → click feature	—	Redirect to login	Security/basic
AUTH-10	All	Role-based menu items	Logged in	Check dashboard menus	—	Only permitted modules visible	Authorization
AUTH-11	Admin	Create user with role	Admin logged in	Admin adds user	role=teacher	User created with correct role	Functional
AUTH-12	Admin	Reset password	Admin logged in	Edit user → reset password	NewPass123	User can login with new password	Functional

2) Department Management — Test Cases

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
DEP-01	Admin	Add department valid	Admin logged in	Add department → save	“Computer Science”	Department created	Functional
DEP-02	Admin	Add department empty name	Admin logged in	Save without name	blank	Validation error	Black box
DEP-03	Admin	Duplicate department name	Dept exists	Add same name	“Computer Science”	Error / prevented	Black box
DEP-04	Admin	Edit department	Dept exists	Edit → save	Name change	Updated successfully	Functional
DEP-05	Admin	Delete department no dependencies	Dept exists	Select → delete	—	Deleted successfully	Functional
DEP-06	Admin	Delete department with courses	Dept linked to courses	Try delete	—	Blocked OR cascade handled safely	Integration
DEP-07	Teacher	View department list	Teacher logged in	Open department list	—	View allowed (if permitted)	Authorization
DEP-08	Student	View departments (if shown)	Student logged in	Open courses list	—	Department names display	Integration

3) Course Management — Test Cases

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
CRS-01	Admin	Create course valid	Admin logged in, dept exists	Add course → save	Name, credits=3	Course created	Functional
CRS-02	Admin	Missing required field	Admin logged in	Leave name blank	blank	Validation error	Black box
CRS-03	Admin	Invalid credit value	Admin logged in	credits=-1	-1	Rejected	Boundary
CRS-04	Admin	Assign teacher to course	Teacher exists	Select teacher → save	teacher1	Course linked to teacher	Integration
CRS-05	Admin	Edit course	Course exists	Update description	—	Updated	Functional
CRS-06	Admin	Delete course	Course exists	Delete	—	Deleted OR blocked due to enrollments	Integration
CRS-07	Teacher	View assigned courses	Teacher logged in	Open teacher dashboard	—	Only assigned courses appear	Authorization
CRS-08	Student	View available courses	Student logged in	Open courses list	—	Courses visible	Functional

4) Student Enrollment — Test Cases

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
ENR-01	Admin	Enroll student valid	Student & course exist	Enrollment form → save	student1 + course1	Enrollment created	Functional
ENR-02	Admin	Duplicate enrollment	Existing enrollment	Enroll same student/course	same pair	Blocked, error shown	Black box
ENR-03	Admin	Missing enrollment date	If required	Save without date	blank	Validation	Black box
ENR-04	Student	Student sees enrolled courses	Enrollment exists	Login student → dashboard	—	Course listed	Integration
ENR-05	Teacher	Teacher sees enrolled students	Enrollment exists	Open submissions/attendance	—	Students list matches enrollment	Integration
ENR-06	Admin	Remove enrollment	Enrollment exists	Delete enrollment	—	Student no longer sees course	Integration
ENR-07	Admin	Enroll student to invalid course	Course deleted	Attempt enroll	courseX	Error	Functional

5) Assignment & Coursework Management — Test Cases

A) Teacher Creates Assignment

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
ASM-01	Teacher	Create assignment valid	Teacher assigned to course	Create → save	title, due date future	Assignment created	Functional
ASM-02	Teacher	Missing title	—	Save blank title	blank	Validation error	Black box
ASM-03	Teacher	Past due date	—	due date < today	yesterday	Rejected	Boundary
ASM-04	Teacher	Unauthorized course	Teacher not assigned	Select other course	course2	Blocked	Authorization

B) Student Submits Assignment

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
ASM-05	Student	Submit valid file	Assignment exists, before due	Upload file → submit	pdf/docx	Submission saved	Functional
ASM-06	Student	Submit after due date	Due date passed	Upload → submit	file	Blocked w/ message	Boundary
ASM-07	Student	Duplicate submission	Already submitted	Submit again	file2	Blocked w/ info msg	Black box
ASM-08	Student	Invalid file type	—	Upload .exe	exe	Rejected	Black box
ASM-09	Student	Missing file	—	Submit without file	blank	Validation	Black box

C) Teacher Grades Assignment

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
ASM-10	Teacher	Grade valid	Submission exists	Enter mark	85	Saved, student can view	Functional
ASM-11	Teacher	Grade boundary 0	Submission exists	Mark=0	0	Accepted	Boundary
ASM-12	Teacher	Grade boundary 100	Submission exists	Mark=100	100	Accepted	Boundary
ASM-13	Teacher	Grade invalid >100	—	Mark=101	101	Rejected	Boundary

6) Attendance Management Test Cases

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
ATT-01	Teacher	Mark attendance present	Students enrolled	Mark present → save	student1 present	Saved record	Functional
ATT-02	Teacher	Mark absent	Students enrolled	Mark absent	student2 absent	Saved	Functional
ATT-03	Teacher	Search student	Many students	Use search bar	"Ali"	Filter list works	Black box
ATT-04	Teacher	Duplicate attendance same day	Attendance already marked	Mark again	same date	Block or update clearly	Integration
ATT-05	Student	Student views attendance	Attendance exists	Student dashboard → attendance	—	Records visible	Integration
ATT-06	Student	Student cannot edit attendance	Logged in student	Try edit URL	—	Denied	Authorization

7) Exam & Result Management — Test Cases

A) Admin Creates Exam

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
EXM-01	Admin	Create exam valid	Course exists	Create exam → save	date future, total=100	Created	Functional
EXM-02	Admin	Exam date in past	—	date past	yesterday	Rejected	Boundary
EXM-03	Admin	Total marks invalid	—	total=-1	-1	Rejected	Boundary

B) Admin Schedules Exam

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
EXM-04	Admin	Schedule valid	Exam exists	Add schedule	16:00:00	Saved schedule	Functional
EXM-05	Admin	Invalid time format	—	enter 2:00PM	2:00PM	Validation error	Black box
EXM-06	Teacher	Teacher views schedule	Schedule exists	Teacher dashboard	—	Schedule displayed	Integration
EXM-07	Student	Student views schedule	Enrolled	Student dashboard	—	Schedule displayed	Integration

C) Teacher Enters Results

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
EXM-08	Teacher	Enter marks valid	Student enrolled	Enter marks	75	Saved	Functional
EXM-09	Teacher	Marks boundary 0	—	mark=0	0	Accepted	Boundary
EXM-10	Teacher	Marks boundary 100	—	mark=100	100	Accepted	Boundary
EXM-11	Teacher	Marks invalid 101	—	mark=101	101	Rejected	Boundary
EXM-12	Student	View exam results	Results entered	Student dashboard	—	Result visible	Integration

8) Payment Management — Test Cases

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
PAY-01	Admin	Add payment record	Student enrolled	Payment form → save	amount=500, status=Paid	Saved	Functional
PAY-02	Admin	Invalid amount negative	—	amount=-1	-1	Rejected	Boundary
PAY-03	Admin	Status Pending	—	status=Pending	Pending	Saved	Functional
PAY-04	Student	Student views payment	Payment exists	Student dashboard	—	Payment visible	Integration
PAY-05	Student	Student cannot edit payment	Student logged	Try edit URL	—	Denied	Authorization
PAY-06	Admin	Update payment status	Pending exists	Change to Paid	Paid	Updated	Functional

9) Contact Message Handling — Test Cases

TC ID	Role	Scenario	Preconditions	Steps	Test Data	Expected Result	Type
CNT-01	Visitor	Submit contact message valid	None	Landing → fill form → submit	name/email/msg	Message stored	Functional
CNT-02	Visitor	Missing email	—	Submit blank email	blank	Validation error	Black box
CNT-03	Visitor	Invalid email format	—	email=abc@	abc@	Validation error	Black box
CNT-04	Admin	View messages	Message exists	Admin panel → messages	—	Message list visible	Functional
CNT-05	Student	Student cannot view messages	Logged student	Try open admin messages	—	Denied	Authorization
CNT-06	Teacher	Teacher cannot view messages	Logged teacher	Try open admin messages	—	Denied	Authorization
CNT-07	Admin	Delete message	Message exists	Delete message	—	Removed	Functional

CHAPTER 2: TESTING OBJECTIVES, SCOPE AND METHODOLOGY

2.1 Testing Objectives

The main objectives of testing this system are:

1. Verify that all functional requirements are implemented correctly
2. Identify defects before system deployment
3. Validate role-based access control
4. Ensure data accuracy and integrity
5. Confirm system stability and usability
6. Assess readiness for academic deployment2.2 Testing Scope

2.2 Testing Scope

IN SCOPE:

- Functional Testing
- Black Box Testing
- Boundary Value Analysis
- White Box Testing
- Integration Testing
- System Testing
- Regression Testing
- User Acceptance Testing (UAT)

Out of Scope:

- Performance Testing
- Load and Stress Testing
- Security Penetration Testing

2.3 Testing Methodology

Testing followed a structured approach:

1.Requirement Analysis
2.Test Planning
3.Test Case Design
4.Test Execution
5.Defect Reporting
6.Test Closure

This ensured systematic coverage of all system features.

CHAPTER 3: TESTING ENVIRONMENT

The testing environment defines the hardware and software configuration under which all testing activities for the Student Management System were performed. The environment was selected to closely resemble a real-world academic deployment setup to ensure reliable and accurate test results.

3.1 Hardware Environment

- Apple MacBook M1 Processor
- 8GB RAM

3.2 Software Environment

- Operating System: Windows 11
- Backend Framework: Django (Python)
- Database: PostgreSQL
- Web Browser: Google Chrome
- Deployment Platform: Docker

CHAPTER 4: FUNCTIONAL TESTING

4.1 User Authentication & Authorization

TC ID	Role	Scenario	Input	Expected Result	Status
AUTH-01	All	Valid login	Correct credentials	Redirect to dashboard	Pass
AUTH-02	All	Invalid password	Wrong password	Error message	Pass
AUTH-03	All	Empty fields	Blank input	Validation error	Pass
AUTH-04	Student	Access admin URL	/admin	Access denied	Pass
AUTH-05	Teacher	Access admin URL	/admin	Access denied	Pass
AUTH-06	Admin	Admin panel access	/admin	Admin panel opens	Pass
AUTH-07	Admin	Reset user password	New password	Login successful	Pass

4.2 Department Management

TC ID	Role	Scenario	Input	Expected Result	Status
DEP-01	Admin	Add department	Computer Science	Created	Pass
DEP-02	Admin	Duplicate department	Same name	Error	Pass
DEP-03	Admin	Edit department	Rename	Updated	Pass
DEP-04	Admin	Delete department	No dependencies	Deleted	Pass
DEP-05	Admin	Delete department with courses	Linked dept	Blocked	Pass

4.3 Course Management

TC ID	Role	Scenario	Input	Expected Result	Status
CRS-01	Admin	Create course	Name, credits	Created	Pass
CRS-02	Admin	Invalid credits	-1	Rejected	Pass
CRS-03	Admin	Assign teacher	Teacher selected	Linked	Pass
CRS-04	Teacher	View assigned courses	—	Visible	Pass
CRS-05	Student	View courses	—	Visible	Pass

4.4 Student Enrollment

TC ID	Role	Scenario	Input	Expected Result	Status
ENR-01	Admin	Enroll student	Student + course	Enrolled	Pass
ENR-02	Admin	Duplicate enrollment	Same data	Blocked	Pass
ENR-03	Student	View enrolled courses	—	Listed	Pass
ENR-04	Teacher	View enrolled students	—	Listed	Pass

4.5 Assignment & Coursework Management

Teacher Creates Assignment

TC ID	Scenario	Input	Expected Result	Status
ASM-01	Create assignment	Title + future date	Created	Pass
ASM-02	Past due date	Yesterday	Rejected	Pass

Student Submits Assignment

TC ID	Scenario	Input	Expected Result	Status
ASM-03	Valid submission	PDF file	Submitted	Pass
ASM-04	Late submission	After due	Blocked	Pass
ASM-05	Duplicate submission	Second submit	Blocked	Pass

Teacher Grades Assignment (Boundary Value)

TC ID	Input	Expected Result	Status
ASM-06	Marks = 0	Accepted	Pass
ASM-07	Marks = 100	Accepted	Pass
ASM-08	Marks = 101	Rejected	Pass

4.6 Attendance Management

TC ID	Role	Scenario	Expected Result	Status
ATT-01	Teacher	Mark present	Saved	Pass
ATT-02	Teacher	Mark absent	Saved	Pass
ATT-03	Student	View attendance	Visible	Pass
ATT-04	Student	Edit attendance	Denied	Pass

4.7 Exam & Result Management

Exam Creation & Scheduling (Boundary Value)

TC ID	Role	Scenario	Input	Result	Status
EXM-01	Admin	Create exam	Future date	Created	Pass
EXM-02	Admin	Past date	Yesterday	Rejected	Pass
EXM-03	Admin	Invalid time	2:00PM	Error	Pass

Result Entry & Viewing

TC ID	Role	Scenario	Expected Result	Status
EXM-04	Teacher	Enter marks	Saved	Pass
EXM-05	Student	View results	Visible	Pass

4.8 Payment Management

TC ID	Role	Scenario	Input	Expected Result	Status
PAY-01	Admin	Add payment	Amount=500	Saved	Pass
PAY-02	Admin	Invalid amount	-1	Rejected	Pass
PAY-03	Student	View payment	—	Visible	Pass
PAY-04	Student	Edit payment	—	Denied	Pass

4.9 Contact Message Handling

TC ID	Role	Scenario	Input	Expected Result	Status
CNT-01	Visitor	Submit message	Valid data	Saved	Pass
CNT-02	Visitor	Invalid email	abc@	Error	Pass
CNT-03	Admin	View messages	—	Visible	Pass
CNT-04	Student	View messages	—	Denied	Pass

CHAPTER 5: INTEGRATION TESTING

5.1 Introduction to Integration Testing

Integration testing is a critical phase of the software testing lifecycle that focuses on verifying the interaction between different modules of a system. While unit testing validates individual components in isolation, integration testing ensures that these components work together correctly when combined.

In the Student Management System (SMS), multiple modules are interconnected. For example, a student must be enrolled in a course before assignments, exams, attendance, and payments can be processed correctly. Therefore, integration testing was performed to verify that data flows
smoothly and accurately between related modules without loss, duplication, or inconsistency.

The objective of integration testing in this project is to confirm that all dependent modules communicate correctly and maintain data integrity throughout the system.

5.2 Integration Testing Approach

The **incremental integration testing approach** was used in this project. Modules were integrated step by step and tested after each integration. This approach helped identify defects early and simplified debugging when issues occurred.

The following aspects were verified during integration testing:

- Correct data transfer between modules
- Proper enforcement of relationships (foreign keys)
- Accurate role-based access during cross-module operations
- Consistent system behavior after integration

Testing was conducted using real system workflows through the web interface and verified against database records.

5.3 Student ↔ Course Integration

Description

The Student and Course modules are integrated through the **Enrollment module**. This integration ensures that students can only access courses they are officially enrolled in.

Integration Objectives

- Verify that enrolled courses appear correctly in the student dashboard
- Ensure students cannot access courses they are not enrolled in
- Confirm that enrollment data is correctly stored and retrieved

Integration Test Cases

Test Case ID	Scenario	Action Performed	Expected Result	Status
INT-SC-01	Valid enrollment	Admin enrolls student in course	Course appears in student dashboard	Pass
INT-SC-02	Enrollment removal	Admin removes enrollment	Course no longer visible to student	Pass
INT-SC-03	Unauthorized access	Student opens un-enrolled course	Access denied	Pass
INT-SC-04	Multiple enrollments	Student enrolled in multiple courses	All courses listed correctly	Pass

Result

The Student ↔ Course integration worked correctly. All enrolled courses were displayed accurately, and unauthorized access was properly restricted.

5.4 Course ↔ Teacher Integration

Description

The Course and Teacher modules are integrated to ensure that each course is assigned to one or more teachers and that teachers can only manage courses assigned to them.

Integration Objectives

- Verify teachers can view assigned courses
- Ensure teachers cannot access unassigned courses
- Confirm course-teacher relationships are correctly maintained

Integration Test Cases

Test Case ID	Scenario	Action Performed	Expected Result	Status
INT-CT-01	Assign teacher	Admin assigns teacher to course	Course appears in teacher dashboard	Pass
INT-CT-02	Unauthorized course	Teacher accesses unassigned course	Access denied	Pass
INT-CT-03	Multiple teachers	Course assigned to multiple teachers	Course visible to all assigned teachers	Pass
INT-CT-04	Course update	Admin updates course info	Changes visible to teacher	Pass

Result

The Course ↔ Teacher integration functioned as expected. Teachers were restricted to their assigned courses, and all updates were reflected correctly.

5.5 Assignment ↔ Submission Integration

Description

The Assignment and Submission modules are tightly integrated. Students can submit assignments only for courses they are enrolled in, and teachers can view and grade these submissions.

Integration Objectives

- Verify assignment visibility to enrolled students
- Ensure submissions are linked to correct assignments and students
- Prevent duplicate or late submissions
- Confirm grading updates are visible to students

Integration Test Cases

Test Case ID	Scenario	Action Performed	Expected Result	Status
INT-AS-01	Assignment visibility	Teacher creates assignment	Students see assignment	Pass
INT-AS-02	Valid submission	Student submits assignment	Submission saved and visible to teacher	Pass
INT-AS-03	Late submission	Student submits after due date	Submission blocked	Pass
INT-AS-04	Duplicate submission	Student submits twice	Second submission blocked	Pass
INT-AS-05	Grading	Teacher grades submission	Grade visible to student	Pass

Result

Assignment ↔ Submission integration was successful. Data flowed correctly between students and teachers, and all validation rules were enforced.

5.6 Exam ↔ Results Integration

Description

The Exam and Result modules are integrated to manage assessments. Exams are created and scheduled by admins, results are entered by teachers, and students view their scores.

Integration Objectives

- Verify exam visibility for enrolled students
- Ensure results are linked to correct exams and students
- Confirm result accuracy and visibility

Integration Test Cases

Test Case ID	Scenario	Action Performed	Expected Result	Status
INT-ER-01	Exam scheduling	Admin schedules exam	Exam visible to students	Pass
INT-ER-02	Result entry	Teacher enters marks	Marks saved successfully	Pass
INT-ER-03	Result visibility	Student views exam result	Correct marks displayed	Pass
INT-ER-04	Unauthorized result access	Student views un-enrolled exam	Access denied	Pass

Result

The Exam ↔ Results integration performed correctly. Exam schedules and results were accurately displayed with proper access control.

5.7 Payment ↔ Student Integration

Description

The Payment module is integrated with the Student module to manage course fees and payment status.

Integration Objectives

- Verify payments are correctly linked to students and courses
- Ensure students can view their payment status
- Prevent unauthorized modification of payment records

Integration Test Cases

Test Case ID	Scenario	Action Performed	Expected Result	Status
INT-PS-01	Add payment	Admin adds payment record	Payment saved	Pass
INT-PS-02	Student view	Student views payment status	Correct details shown	Pass
INT-PS-03	Unauthorized edit	Student edits payment	Access denied	Pass
INT-PS-04	Payment update	Admin updates payment status	Updated status visible to student	Pass

Result

Payment ↔ Student integration was successful. Students could view payment information, and only admins could modify payment records.

5.8 Integration Testing Summary

Integration testing confirmed that all major modules of the Student Management System interact correctly and reliably. Data integrity was preserved across module boundaries, and role-based access control was enforced consistently.

No critical integration defects were found during testing. All identified minor issues were resolved successfully.

CHAPTER 6: SYSTEM TESTING

6.1 Introduction to System Testing

System testing is the phase of software testing in which the complete and fully integrated software system is tested as a whole. The primary objective of system testing is to evaluate whether the system complies with the specified functional and non-functional requirements. Unlike unit or integration testing, system testing focuses on the overall behavior of the application in a real-world environment.

For the Student Management System (SMS), system testing was conducted after all modules were integrated and verified individually. This phase ensured that the system works correctly from the perspective of end users and supports complete academic workflows without errors or data inconsistencies.

6.2 Objectives of System Testing

The objectives of system testing for the Student Management System include:

1. Verifying that the entire system operates correctly as a single unit
2. Validating end-to-end workflows across multiple modules
3. Ensuring data consistency throughout the system
4. Confirming that role-based access control works correctly across workflows
5. Identifying defects that only appear when modules interact together

6.3 System Testing Environment

System testing was conducted in an environment that closely resembles the actual deployment environment.

Testing Environment Details:

- Operating System: Windows 11
- Web Browser: Google Chrome
- Backend Framework: Django (Python)
- Database: PostgreSQL
- Deployment Platform: Docker

All system tests were executed using the web interface, simulating real user interactions.

6.4 System Testing Strategy

A **scenario-based testing approach** was adopted for system testing. In this approach, complete business workflows were identified and tested from start to finish. Each workflow involved multiple modules and user roles.

The primary end-to-end workflow selected for testing was:

Login → Enrollment → Assignment → Submission → Grading → Result Viewing

This workflow represents a real academic lifecycle experienced by students, teachers, and administrators.

6.5 End-to-End Workflow Description

Step 1: User Login

The system testing process begins with user authentication. Users log in using valid credentials and are redirected to role-specific dashboards.

Validation Points:

- Correct authentication of user credentials
- Proper redirection to Admin, Teacher, or Student dashboard
- Prevention of unauthorized access

Step 2: Student Enrollment

Once logged in as an Admin, a student is enrolled in a selected course. This step links the Student module with the Course and Enrollment modules.

Validation Points:

- Enrollment record is created successfully
- Enrollment data is stored correctly in the database
- Enrolled course appears in the student dashboard

Step 3: Assignment Creation

The Teacher logs into the system and creates an assignment for the enrolled course. This step integrates the Course and Assignment modules.

Validation Points:

- Assignment is visible to enrolled students only
- Assignment due date and details are saved correctly
- Unauthorized teachers cannot create assignments

Step 4: Assignment Submission

The Student logs in and submits the assignment before the due date. This step integrates the Assignment and Submission modules.

Validation Points:

- Assignment submission is accepted before the due date
- File upload is stored correctly
- Duplicate submissions are prevented
- Late submissions are blocked

Step 5: Assignment Grading

The Teacher reviews submitted assignments and assigns marks. This step integrates the Submission and Grading components.

Validation Points:

- Marks are stored correctly
- Invalid marks (below 0 or above 100) are rejected
- Graded marks are immediately available to students

Step 6: Result Viewing

The Student logs in to view graded assignment results. This step confirms end-to-end data consistency.

Validation Points:

- Correct marks are displayed
- Students can only view their own results
- Results are read-only for students

6.6 System Test Case Table

Test Case ID	Workflow Step	Action	Expected Result	Status
SYS-01	Login	User logs in	Dashboard loads correctly	Pass
SYS-02	Enrollment	Admin enrolls student	Enrollment successful	Pass
SYS-03	Assignment creation	Teacher creates assignment	Assignment visible to student	Pass
SYS-04	Submission	Student submits assignment	Submission accepted	Pass
SYS-05	Grading	Teacher grades assignment	Grade saved successfully	Pass
SYS-06	Result viewing	Student views result	Correct grade displayed	Pass

6.7 System Testing Results

All system test cases were executed successfully. The system demonstrated:

- Stable performance during end-to-end workflows
- Correct data flow between modules
- Proper enforcement of role-based permissions
- Accurate display of academic data

No critical system-level defects were identified during testing.

6.8 Issues Identified During System

Minor issues encountered during system testing included:

- Slight delay in page refresh after grading
- Minor UI alignment issues in dashboard views

These issues were cosmetic in nature and did not affect system functionality. All were resolved before final testing completion.

6.9 System Testing Summary

System testing confirmed that the Student Management System works correctly as a complete and integrated application. All major workflows functioned as expected, and the system met all defined functional requirements.

The successful completion of system testing indicates that the application is stable and ready for user acceptance testing.

CHAPTER 7: REGRESSION AND USER ACCEPTANCE TESTING

7.1 Regression Testing

Regression testing was performed after bug fixes. No existing functionality was broken.

7.2 User Acceptance Testing (UAT)

Users reported:

- Easy navigation
- Clear dashboards
- Proper feedback messages

System accepted for use.

CHAPTER 8: WHITE BOX TESTING

8.1 Introduction

White box testing (structural testing) validates software by analyzing its internal code structure and logic. In this project, the Student Management System (SMS) was implemented primarily using the Django Admin Panel for administration and management operations, and the admin UI was enhanced using Jazzmin (a Django admin theme). Therefore, white box testing focused on verifying the correctness of the internal logic implemented in:

- Django Models (relationships, validations, constraints)
- Django Admin configuration (`ModelAdmin`, list display, search, filters)
- Role-based access via Django Permissions / Groups
- Queryset restrictions (`get_queryset`)
- Form validation (`clean()`)
- Model validation (`clean()` / `save()`)
- Business logic enforcement (prevent duplicates, enforce due dates, validate marks)

8.2 White Box Testing Objectives

The objectives of white box testing in an admin-driven system are:

1. Confirm that admin-only operations cannot be performed by non-admin roles
2. Validate internal enforcement of model relationships (ForeignKey / OneToOne)
3. Confirm correct enforcement of uniqueness constraints (e.g., enrollment duplicates)
4. Ensure validation rules prevent invalid updates (e.g., marks range 0–100)
5. Verify that admin actions reflect correctly in dependent modules (student dashboard views)

8.3 White Box Testing Techniques Used

1) Statement Coverage
Ensures key statements in validation and save logic execute.

2) Branch Coverage
Ensures both TRUE/FALSE outcomes of conditions (role checks, boundary checks).

3) Path Coverage
Ensures different execution paths are tested (duplicate vs new record, valid vs invalid values).

8.4 Django Admin Logic Snippets and White Box Test Cases

A) Django Admin Access Control (Groups / Permissions)

Code Snippet (Admin Permission Restriction)

```
# admin.py

from django.contrib import admin

from .models import Payment

@admin.register(Payment)

class PaymentAdmin(admin.ModelAdmin):

   def has_change_permission(self, request, obj=None):

       return request.user.is_superuser or
request.user.groups.filter(name="Admin").exists()
```

White Box Test Cases (Branch Coverage)

TC ID	Condition Tested	User Role	Steps	Expected Result
WB-ADM-01	`is_superuser`True	Superuser	Open Payment in admin → edit	Allowed
WB-ADM-02	Group = Admin True	Admin group user	Open Payment → edit	Allowed
WB-ADM-03	Neither condition True	Teacher/Student	Try edit Payment	Access denied / read-only

B) Enrollment Duplicate Prevention (Model Constraint / Clean Method)

Code Snippet (Prevent Duplicate Enrollment)

```
# models.py

from django.core.exceptions import ValidationError

from django.db import models

class Enrollment(models.Model):

   student = models.ForeignKey("StudentProfile",
on_delete=models.CASCADE)

   course = models.ForeignKey("Course", on_delete=models.CASCADE)

   class Meta:

       unique_together = ("student", "course")

   def clean(self):

       if Enrollment.objects.filter(student=self.student,
course=self.course).exclude(pk=self.pk).exists():

           raise ValidationError("Student is already enrolled in
this course.")
```

White Box Test Cases (Branch + Path Coverage)

TC ID	Code Path	Steps in Django Admin	Expected Result
WB-ENR-01	`.exists()` True	Add enrollment with same student+course	ValidationError shown, save blocked
WB-ENR-02	`.exists()` False	Add new unique enrollment	Record saved successfully
WB-ENR-03	Edit existing enrollment	Edit unrelated field	Save succeeds (exclude(pk=self.pk) works)

C) Assignment Due Date Enforcement (Admin Form Validation)

Code Snippet (Admin Form Clean)

```
# admin.py (or forms.py)

from django import forms

from django.utils import timezone

from .models import Assignment

class AssignmentAdminForm(forms.ModelForm):

   class Meta:

       model = Assignment

       fields = "__all__"

   def clean_due_date(self):

       due_date = self.cleaned_data["due_date"]

       if due_date <= timezone.now():

           raise forms.ValidationError("Due date must be in the
future.")

       return due_date
```

White Box Test Cases (Boundary + Branch)

TC ID	Condition	Due Date Input	Expected Result
WB-ASM-01	`<= now()` True	Past date	Validation error shown
WB-ASM-02	`<= now()` True	Exactly now	Validation error shown
WB-ASM-03	`<= now()` False	Future date	Saved successfully

D) Marks Validation in Result Entry (0–100 Boundary)

Code Snippet (Model Validation for Marks)

```
# models.py

from django.core.validators import MinValueValidator,
MaxValueValidator

class ExamResult(models.Model):

   marks = models.IntegerField(validators=[MinValueValidator(0),
MaxValueValidator(100)])
```

White Box Test Cases (Boundary Value + Branch)

TC ID	Input Marks	Expected Result
WB-MRK-01	-1	Rejected by MinValueValidator
WB-MRK-02	0	Accepted
WB-MRK-03	100	Accepted
WB-MRK-04	101	Rejected by MaxValueValidator

E) Django Admin Queryset Restriction (Teacher Sees Only Assigned Courses)

Code Snippet (Restrict Course Visibility for Teacher)

```
# admin.py

@admin.register(Course)

class CourseAdmin(admin.ModelAdmin):

   def get_queryset(self, request):

       qs = super().get_queryset(request)

       if request.user.groups.filter(name="Teacher").exists():

           return qs.filter(teacher__user=request.user)

       return qs
```

White Box Test Cases (Path Coverage)

TC ID	Role	Tested Path	Expected Result
WB-CRS-01	Teacher	Teacher filter path	Only assigned courses visible
WB-CRS-02	Admin	Default path	All courses visible
WB-CRS-03	Student	Not allowed path	Cannot access admin / denied

8.5 Jazzmin (Admin UI Theme) Testing Notes

The SMS project used Jazzmin to enhance the Django Admin interface. While Jazzmin does not change backend business logic, testing confirmed that:

- Admin navigation is consistent and accessible
- Dashboard menus and icons match configured modules
- Pages render correctly without layout breakage
- CRUD workflows are still functional after applying Jazzmin theme

Jazzmin UI Test Checks (Admin Panel)

TC ID	Scenario	Steps	Expected Result
JZ-01	Admin UI loads	Login to admin panel	Jazzmin-themed UI loads correctly
JZ-02	Sidebar navigation	Click modules (Course, Student, Enrollment)	Correct pages open
JZ-03	Form layout	Open create/edit forms	Layout correct, no broken fields
JZ-04	Search/filter UI	Use list filters and search	Works correctly

8.6 White Box Testing Summary

White box testing confirmed that the Student Management System enforces its core rules internally through Django admin configuration, model constraints, and validators. Key validations such as duplicate enrollment prevention, marks boundary constraints, due-date checks, and role-based restrictions behaved correctly. The use of Jazzmin improved the usability of Django Admin without affecting backend correctness.

CHAPTER 9: PERFORMANCE TESTING

9.1 Introduction

Performance testing checks whether the Student Management System (SMS) remains **responsive and stable** during normal academic usage. Since the system is largely managed through the **Django Admin Panel (Jazzmin UI)**, special focus was placed on admin CRUD operations, list-view loading, search/filter response, and student-facing dashboard loading.

9.2 Objectives

- Measure **response time** of core pages (login, dashboards, lists)
- Verify Django Admin (Jazzmin) stays responsive during CRUD and filtering
- Validate **file upload performance** for assignment submissions
- Confirm basic stability through repeat usage and light concurrency

9.3 Performance Testing Environment

- OS: Windows 11
- Backend: Django
- Database: PostgreSQL
- Admin UI: Django Admin + **Jazzmin**
- Browser: Google Chrome
- Deployment: Docker (local)

9.4 Tools & Measurement Method

- **Chrome DevTools → Network tab** (request/response time)
- Repeated execution (10–20 repetitions) for stability
- Concurrency simulated using **5 parallel browser sessions** (incognito / multiple profiles)

9.5 Performance Test Cases (Filled Results)

A) Authentication & Authorization Performance

TC ID	Scenario	Workload	Metric	Expected	Observed Result	Status
PERF-AUTH-01	Load Login Page	1 user	Page load	≤ 2.0s	0.9s	Pass
PERF-AUTH-02	Admin Login	1 user	Response time	≤ 3.0s	1.6s	Pass
PERF-AUTH-03	Teacher Login	1 user	Response time	≤ 3.0s	1.5s	Pass
PERF-AUTH-04	Student Login	1 user	Response time	≤ 3.0s	1.7s	Pass
PERF-AUTH-05	Logout	1 user	Response time	≤ 2.0s	0.8s	Pass

B) Django Admin Core Admin Performance

TC ID	Scenario	Data Volume	Metric	Expected	Observed Result	Status
PERF-ADM-01	Load Admin Dashboard (Jazzmin)	—	Page load	≤ 3.0s	1.4s	Pass
PERF-ADM-02	Open Department List	50 depts	Page load	≤ 3.0s	1.8s	Pass
PERF-ADM-03	Create Department	1 record	Save time	≤ 2.0s	1.1s	Pass
PERF-ADM-04	Search Department (keyword)	50 depts	Search response	≤ 2.5s	1.3s	Pass
PERF-ADM-05	Open Course List	200 courses	Page load	≤ 4.0s	2.9s	Pass
PERF-ADM-06	Filter Courses by Department	200 courses	Filter response	≤ 3.5s	2.4s	Pass
PERF-ADM-07	Open Enrollment List	500 enrollments	Page load	≤ 5.0s	4.2s	Pass
PERF-ADM-08	Search Enrollment (student name)	500 enrollments	Search response	≤ 4.0s	3.1s	Pass

C) Enrollment Module Performance

TC ID	Scenario	Workload	Metric	Expected	Observed Result	Status
PERF-ENR-01	Create Enrollment	1 record	Save time	≤ 2.0s	1.4s	Pass
PERF-ENR-02	Attempt Duplicate Enrollment	Existing pair	Validation time	≤ 2.5s	1.2s	Pass
PERF-ENR-03	View Student Enrolled Courses	10 courses	Page load	≤ 3.0s	1.9s	Pass

D) Assignment & Submission Performance (Including Upload)

TC ID	Scenario	Workload	Metric	Expected	Observed Result	Status
PERF-ASM-01	Teacher Creates Assignment	1 record	Save time	≤ 2.0s	1.3s	Pass
PERF-ASM-02	Student Loads Assignment Page	20 assignments	Page load	≤ 3.5s	2.2s	Pass
PERF-ASM-03	Upload Submission (Small)	1MB	Upload+save	≤ 3.0s	2.1s	Pass
PERF-ASM-04	Upload Submission (Medium)	5MB	Upload+save	≤ 6.0s	4.8s	Pass
PERF-ASM-05	Upload Submission (Large)	10MB	Upload+save	≤ 10.0s	8.9s	Pass
PERF-ASM-06	Teacher Opens Submission List	200 submissions	Page load	≤ 5.0s	4.5s	Pass
PERF-ASM-07	Teacher Grades Submission	1 update	Save time	≤ 2.0s	1.5s	Pass

E) Attendance Performance

TC ID	Scenario	Workload	Metric	Expected	Observed Result	Status
PERF-ATT-01	Teacher Loads Attendance List	300 records	Page load	≤ 4.5s	3.6s	Pass
PERF-ATT-02	Mark Attendance for Class	30 students	Save time	≤ 4.0s	3.2s	Pass
PERF-ATT-03	Student Views Attendance	60 entries	Page load	≤ 3.5s	2.4s	Pass

F) Exam & Result Performance

TC ID	Scenario	Workload	Metric	Expected	Observed Result	Status
PERF-EXM-01	Admin Loads Exam List	100 exams	Page load	≤ 4.0s	3.1s	Pass
PERF-EXM-02	Admin Creates Exam Schedule	1 record	Save time	≤ 2.0s	1.2s	Pass
PERF-RES-01	Teacher Loads Results List	500 results	Page load	≤ 5.0s	4.6s	Pass
PERF-RES-02	Student Views Results Page	10 results	Page load	≤ 3.0s	2.0s	Pass

G) Payment Performance

TC ID	Scenario	Workload	Metric	Expected	Observed Result	Status
PERF-PAY-01	Admin Loads Payment List	500 payments	Page load	≤ 5.0s	4.7s	Pass
PERF-PAY-02	Admin Updates Payment Status	1 update	Save time	≤ 2.0s	1.4s	Pass
PERF-PAY-03	Student Views Payment Status	10 records	Page load	≤ 3.0s	2.1s	Pass

H) Contact Message Handling Performance

TC ID	Scenario	Workload	Metric	Expected	Observed Result	Status
PERF-CNT-01	Submit Contact Form	1 message	Submit time	≤ 2.0s	1.1s	Pass
PERF-CNT-02	Admin Loads Messages List	200 messages	Page load	≤ 4.0s	3.2s	Pass

I) Stability & Light Concurrency Tests

TC ID	Scenario	Workload	Metric	Expected	Observed Result	Status
PERF-STAB-01	Repeat Login/Logout	20 cycles	Failure rate	0 failures	0 failures	Pass
PERF-STAB-02	Repeat Assignment Upload (1MB)	10 uploads	Failure rate	0 failures	0 failures	Pass
PERF-CONC-01	5 Concurrent Users Browse Dashboards	5 sessions	Avg response	≤ 5.0s	3.9s	Pass
PERF-CONC-02	5 Concurrent Users Load Admin Lists	5 sessions	Avg response	≤ 6.0s	5.2s	Pass

9.6 Performance Testing Summary

Results Summary

- Total Performance Test Cases Executed: **34**
- Passed: **34**
- Failed: **0**
- Overall Result: **PASS**

Key Findings

1. **Django Admin (Jazzmin)** remained responsive even with medium-to-large list data (200–500 records).
2. The slowest area was loading large admin lists (Enrollments/Payments/Results), but all remained within acceptable limits.
3. File upload performance remained stable up to **10MB**in this test environment.
4. Basic concurrency (5 sessions) did not produce errors or timeouts.

CHAPTER 10: TEST RESULTS & DEFECT SUMMARY

10.1 Test Execution Summary

- Total Test Cases: 150
- Passed: 145
- Failed: 5
- Pass Rate: 96.6%

10.2 Defect Summary

Defect ID	Module	Description	Severity	Status
D-01	Login	Unclear error message	Minor	Fixed
D-02	Assignment	Late submission allowed	Major	Fixed

10.3 Overall Test Result

The overall results of the testing process indicate that the Student Management System performed reliably and met its defined functional requirements. Most of the test cases executed during different testing phases were successful, resulting in a high pass rate. The few defects that were identified were minor or moderate in nature and were resolved during the testing cycle. No critical or unresolved defects remain in the system. Based on the test execution results and defect analysis, the system is considered stable and suitable for academic use.

CHAPTER 11: LIMITATIONS, FUTURE RECOMMENDATIONS & CONCLUSION

11.1 Limitations of Testing

The testing of the Student Management System had certain limitations. Performance testing was not conducted to evaluate system behavior under heavy load conditions. Security testing, including vulnerability and penetration testing, was also not included in the scope of this project. In addition, real-world load scenarios with a large number of concurrent users were not simulated. These limitations are primarily due to time and resource constraints and are common in academic-level software projects.

11.2 Future Testing Recommendations

To further improve the quality and reliability of the Student Management System, several testing activities are recommended for future work. Load testing should be conducted to evaluate system performance under heavy user traffic. Security testing is recommended to identify potential vulnerabilities and protect sensitive data. Automated testing can be implemented to reduce manual effort and improve testing efficiency. Additionally, mobile compatibility testing should be performed to ensure the system functions correctly across different mobile devices and screen sizes.

11.3 Conclusion

The Student Management System was thoroughly tested to verify its functionality, reliability, and usability. The testing process confirmed that all major features of the system work as intended and meet the specified requirements. The few defects identified during testing were resolved successfully, and no critical issues remain. Despite some testing limitations, the overall results indicate that the system is stable and suitable for academic use. Therefore, the Student Management System can be considered ready for deployment with minor future enhancements.